発展プログラミング (Advanced Programming)

4. デザインパターンの応用

尾崎 知伸 ＜ozaki.tomonobu@nihon-u.ac.jp＞

Bridge パターン
～機能の階層と実装の階層を分ける～

Bridgeパターン

- Bridgeパターン：「機能」のクラス階層と「実装」のクラス階層を橋渡しする
  - 機能＝メソッド
  - 実装＝抽象メソッドの具体化
- 抽象クラスの継承時に，「機能追加」と「実装」を同時に行うとコードの見通しが悪くなる
  - 継承の目的・意味が複雑・難解になる

```
abstract class Display {
    public abstract void open();
    public abstract void print();
    public abstract void close();
    public final void display() {
        open();
        print();
        close();
    }
}
```

継承

```
class StringDisplay extends Display {
    private String str;
    public StringDisplay(String str) {
        this.str = str;
    }
    public void open() {
        printLine();
    }
    public void print() {
        System.out.println("|" + str + "|");
    }
    public void close() {
        printLine();
    }
    private void printLine() {
        System.out.print("+");
        for (int i = 0; i < str.length(); i++) {
            System.out.print("-");
        }
        System.out.println("+");
    }
    public void multiDisplay(int times) {
        open();
        for (int i = 0; i < times; i++) {
            print();
        }
        close();
    }
}
```

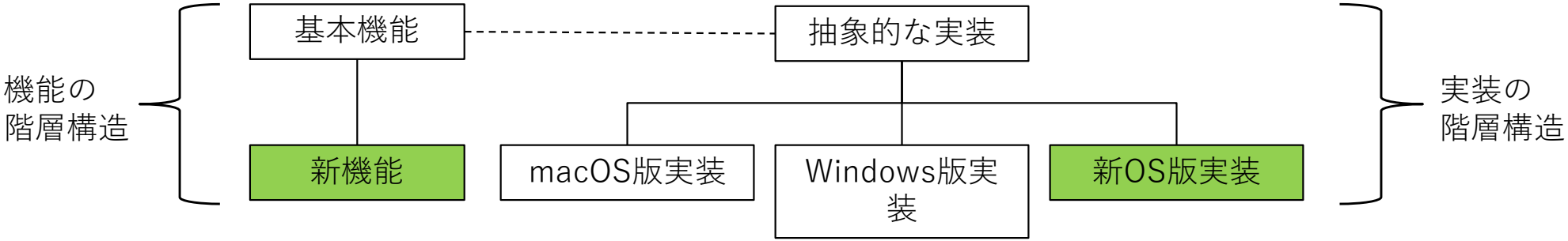
実装

機能の追加

Bridgeパターン

利用シーン

- 機能，実装とも今後拡張が予想される場合
  → 機能の拡張と，実装の拡張が，混在せずに済む
  → 例えば，各OS共通の機能は，「機能の階層」を拡張する
  → 新OSに対応する場合は，「実装の階層」を拡張する

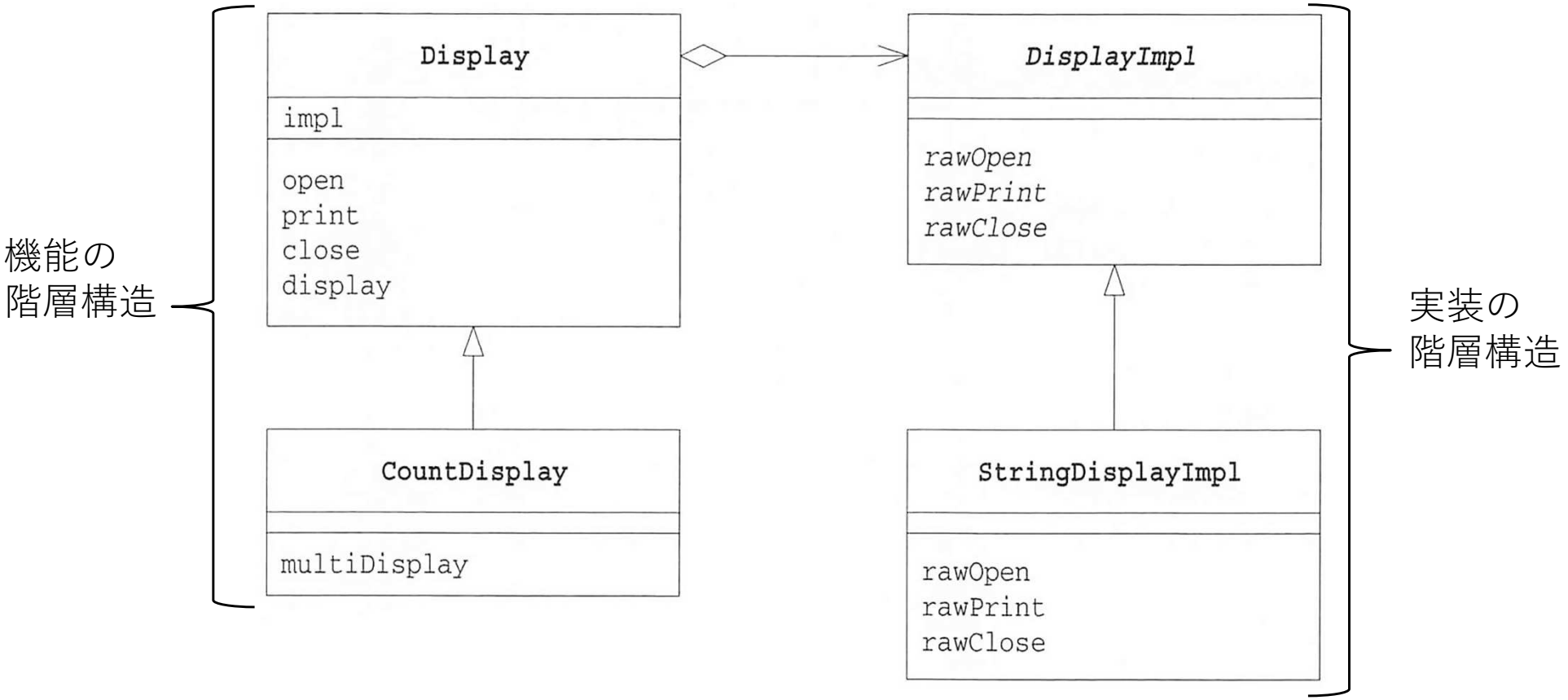


サンプルコード

lec04_bridge/*.java

コンパイル方法: javac lec04_bridge/Main.java
実行方法: java lec04_bridge/Main

橋のどちら側？	名　前	解　説
機能のクラス階層	Display	「表示する」クラス
機能のクラス階層	CountDisplay	「指定回数だけ表示する」という機能を追加したクラス
実装のクラス階層	DisplayImpl	「表示する」クラス
実装のクラス階層	StringDisplayImpl	「文字列を使って表示する」クラス
	Main	動作テスト用のクラス



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

```java
public class Display {
    private DisplayImpl impl;
    public Display(DisplayImpl impl) {
        this.impl = impl;
    }
    public void open() {
        impl.rawOpen();
    }
    public void print() {
        impl.rawPrint();
    }
    public void close() {
        impl.rawClose();
    }
    public final void display() {
        open();
        print();
        close();
    }
}
```

```java
public abstract class DisplayImpl {
    public abstract void rawOpen();
    public abstract void rawPrint();
    public abstract void rawClose();
}
```

impl を通じて各機能を実現
implの実体を知る必要なし
→implの新たな実装が増えても
Displayを変更する必要なし

実装を追加するための継承
（抽象メソッドが実装された）

機能を追加するための継承
（multiDisplayメソッドが追加された）

```java
public class CountDisplay extends Display {
    public CountDisplay(DisplayImpl impl) {
        super(impl);
    }
    public void multiDisplay(int times) {
        open();
        for (int i = 0; i < times; i++) {
            print();
        }
        close();
    }
}
```

```java
public class StringDisplayImpl extends DisplayImpl {
    private String string;
    private int width;
    public StringDisplayImpl(String string) {
        this.string = string;
        this.width = string.getBytes().length;
    }
    public void rawOpen() {
        printLine();
    }
    public void rawPrint() {
        System.out.println("|" + string + "|");
    }
    public void rawClose() {
        printLine();
    }
    private void printLine() {
        System.out.print("+");
        for (int i = 0; i < width; i++) {
            System.out.print("-");
        }
        System.out.println("+");
    }
}
```

機能の階層

実装の階層

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

lec04_bridge/*.java

コンパイル方法: javac lec04_bridge/Main.java
実行方法: java lec04_bridge/Main

```
public class Main {
    public static void main(String[] args) {
        Display d1 = new Display(new StringDisplayImpl("Hello, Japan."));
        Display d2 = new CountDisplay(new StringDisplayImpl("Hello, World."));
        CountDisplay d3 = new CountDisplay(new StringDisplayImpl("Hello, Universe."));
        d1.display();
        d2.display();
        d3.display();
        d3.multiDisplay(5);
    }
}
```

Display (CountDisplay)に対し，DisplayImplの実体を渡す

d1.display();

```
+-------------+
|Hello, Japan.|
+-------------+
```

d2.display();

```
+-------------+
|Hello, World.|
+-------------+
```

d3.display();

```
+----------------+
|Hello, Universe.|
+----------------+
```

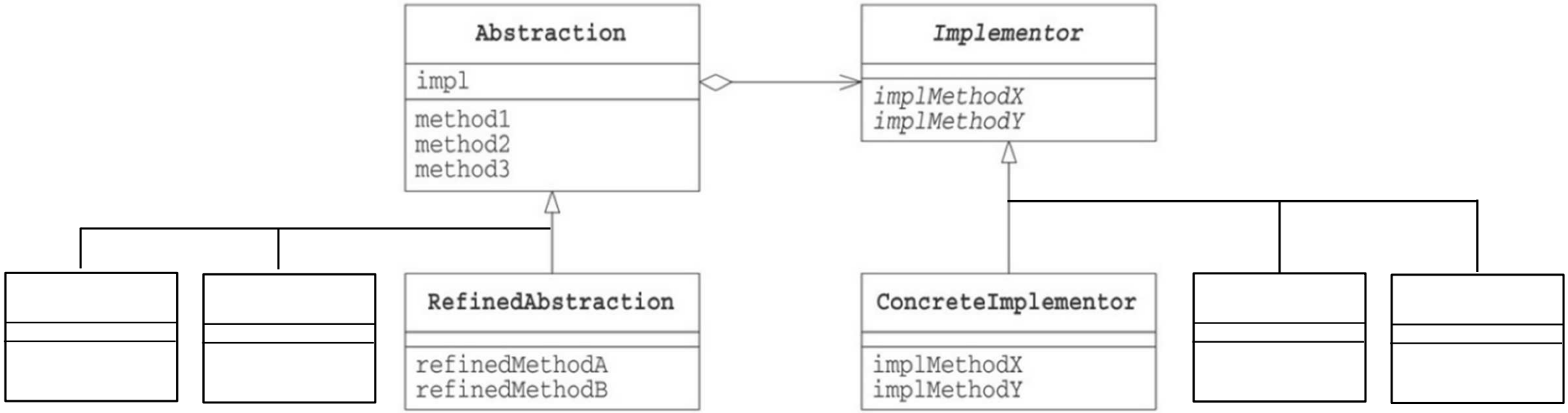
d3.multiDisplay(5);

```
+----------------+
|Hello, Universe.|
|Hello, Universe.|
|Hello, Universe.|
|Hello, Universe.|
|Hello, Universe.|
+----------------+
```

Bridgeパターンの登場人物

種々のRefinedAbstraction役が種々のConcreteImplementorを利用可能

- Abstraction（抽象化）の役
  - 「機能のクラス階層」の最上位のクラス
  - Implementor役のインスタンスを保持
  - Implementor役のメソッドを利用した基本的な機能だけを記述
- RefinedAbstraction（改善した抽象化）の役
  - Abstractionに対して機能を追加した役
  - 同役のクラスは複数追加が可能 → 種々の方面への機能拡張
- Implementor（実装者）の役
  - 「実装のクラス階層」の最上位のクラス
  - Abstraction役のインタフェースを実装するためのメソッドを規定する
- ConcreteImplementor（具体的な実装者）の役
  - 具体的にImplementor役のインタフェースを実装する役・複数の追加が可能



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考えを広げるためのヒント

- 分けておけば，拡張するのが楽になる
  - 「機能のクラス階層」と「実装のクラス階層」の分離
  - それぞれを独立に拡張することができる
    - 実装のクラス階層を変更することなく，機能の追加が可能
    - 機能のクラス階層を変更することなく，実装の追加が可能

- 継承は固い結びつき，委譲は緩やかな結びつき
  - 継承はクラスの拡張には便利．その一方でクラス間のつながりを固定
  - 委譲による結びつき＝委譲先のインスタンスを持つ
    - 委譲先のクラス（インタフェース）の実装を気にしない
    - 委譲先を変えることで，種々の実装を利用することが可能

演習1

- lec04_bridge/*.java
- 下記を実現するために必要なプログラムを追加せよ．またMainを変更し，正しく動作することを確認せよ．
  - 指定した文字列を，ランダム回数表示する
    - （cf. multiDisplay(n) はn回の表示）
  - 指定した文字列配列を，Latexのitemize環境で箇条書き表示する
  - 指定した文字列配列を，HTMLのOL環境で箇条書き表示する
  - どちらの階層にクラスを追加するのか十分考えましょう
  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec04_bridge;
- （余力のある学生さん向け）Bridgeパターンを適用したプログラム例を作成せよ

■乱数作成方法
import java.util.Random; をした上で
Random rand = new Random();
rand.nextInt(10);
とすると0以上10未満の整数乱数が生成できる。

Composite パターン
～容器と中身の同一視～

Composite パターン

- Compositeパターン：
  - 容器（ディレクトリ）と中身（ファイル）を同一視したい（同様に扱いたい）
    - 同様に扱う＝同じ操作（名称・サイズの取得など）をしたい

- ディレクトリ（フォルダ）とファイル
  - ディレクトリの中には，ディレクトリかファイルが入る
  - 入れ子構造，再帰的な構造

- ディレクトリとファイルをまとめて「ディレクトリエントリ」と呼ぶ
  - 両者（ディレクトリとファイル）を同じ種類のものとみなす（同一視する）
  - ディレクトリとファイルは（ある観点で）同じ種類！
    - ディレクトリは，あるディレクトリの中に入れることができる
    - 　　ファイルも，あるディレクトリの中に入れることができる

- 例：ディレクトリの中に何があるか調べる
  - ディレクトリ内は，（サブ）ディレクトリとファイルがある
  - ＝ディレクトリ内は，ディレクトリエントリがある

1つの利点：容器と中身を判別するif文は不要
（Stateパターンと同様）

再帰的な構造を持つ対象をモデル化する
再帰構造の例：線形リスト，ツリーなど

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

サンプルプログラム

lec04_composite/*.java

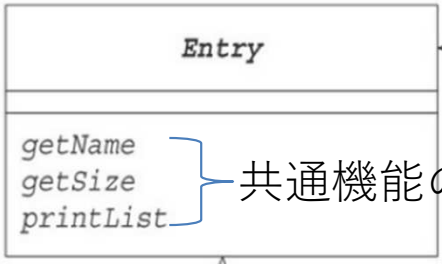
コンパイル方法: javac lec04_composite/Main.java
実行方法: java lec04_composite/Main

- ファイルとディレクトリを模式的に表現

名　前	解　説
Entry	FileとDirectoryを同一視する抽象クラス
File	ファイルを表すクラス
Directory	ディレクトリを表すクラス

共通機能を定義する

役割分担に注目しよう
「抽象クラス＋具体クラス」
（頻出する形です）



共通機能の定義

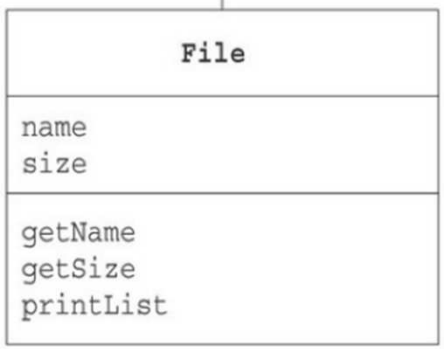
継承

再帰構造の実現

DirectoryがEntryを持つ
Entry は，FileとDirectoryを同一視したもの

File, Directoryを判別するif文が不要

File, Directoryは
具体クラス





所持

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Entryを表すクラス

lec04_composite/*.java

```java
public abstract class Entry {

    public abstract String getName();
    public abstract int getSize();

    protected abstract void printList(String prefix);

    public Entry add(Entry entry) throws FileTreatmentException {
        throw new FileTreatmentException();
    }
    public void printList() {
        printList("");
    }
    public String toString() {
        return getName() + " (" + getSize() + ")";
    }
}
```

前提：名前とサイズを持つ

未学修内容
Entryを追加しようとすると例外を発生させる
（Directoryでは
Overrideする）

オーバーロード

抽象メソッドを含む具体メソッド
（Template Methodパターンの形）

Fileを表すクラス

```java
public class File extends Entry {

    private String name;
    private int size;

    public File(String name, int size) {
        this.name = name;
        this.size = size;
    }
    public String getName() {
        return name;
    }
    public int getSize() {
        return size;
    }
    protected void printList(String prefix) {
        System.out.println(prefix + "/" + this);
    }
}
```

名前とサイズ

Entryを継承

コンストラクタ（name,sizeを指定する）
thisは自分

抽象メソッドｘ3の実装
（実装しなければならない）

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Directoryを表すクラス

lec04_composite/*.java

```java
public class Directory extends Entry {
    private String name;                            // ディレクトリの名前
    private ArrayList<Entry> directory = new ArrayList<Entry>();
    public Directory(String name) {                 // コンストラクタ
        this.name = name;
    }

    public String getName() {                       // 名前を得る
        return name;
    }
    public int getSize() {                          // サイズを得る
        int size = 0;
        Iterator<Entry> it = directory.iterator();
        while ( it.hasNext() ) {
            Entry entry = it.next();
            size += entry.getSize();
        }
        return size;
    }
    public Entry add(Entry entry) {                 // エントリの追加
        directory.add(entry);
        return this;
    }
    protected void printList(String prefix) {       // エントリの一覧
        System.out.println(prefix + "/" + this);
        Iterator<Entry> it = directory.iterator();
        while ( it.hasNext() ) {
            Entry entry = it.next();
            entry.printList(prefix + "/" + name);
        }
    }
}
```

Directoryが
Entryを持つ
＝再帰構造

名前を指定して生成

Entryを保持するリストを使う
（DirectoryもFileもEntry）

Entryとして
容器（Directory）と
中身（File）を
同一視している

サイズ=サブエントリのサイズの合計

FileかDirectoryかは気にしない．If文は不要

オーバーライドで挙動を変える

thisを文字列にする
＝toString()メソッドの結果

所持しているEntryのメソッドを呼び出す（再帰）
※ File かDirectoryかを判断するif文は存在しない

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

lec04_composite/*.java

コンパイル方法: javac lec04_composite/Main.java
実行方法: java lec04_composite/Main

Mainクラス

```java
public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Making root entries...");
            Directory rootdir = new Directory("root");
            Directory bindir = new Directory("bin");
            Directory tmpdir = new Directory("tmp");
            Directory usrdir = new Directory("usr");
            rootdir.add(bindir);
            rootdir.add(tmpdir);
            rootdir.add(usrdir);
            bindir.add(new File("vi", 10000));
            bindir.add(new File("latex", 20000));
            rootdir.printList();

            System.out.println("");
            System.out.println("Making user entries...");
            Directory yuki = new Directory("yuki");
            Directory hanako = new Directory("hanako");
            Directory tomura = new Directory("tomura");
            usrdir.add(yuki);
            usrdir.add(hanako);
            usrdir.add(tomura);
            yuki.add(new File("diary.html", 100));
            yuki.add(new File("Composite.java", 200));
            hanako.add(new File("memo.tex", 300));
            tomura.add(new File("game.doc", 400));
            tomura.add(new File("junk.mail", 500));
            rootdir.printList();
        } catch (FileTreatmentException e) {
            e.printStackTrace();
        }
    }
}
```

例外を表すクラス（未学修内容，理解する必要なし）

```java
/* 例外：未学修内容です */
public class FileTreatmentException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    public FileTreatmentException() {
        super();
    }
    public FileTreatmentException(String msg) {
        super(msg);
    }
}
```

ディレクトリ構成

```
root/
├──bin
│   ├──latex
│   └──vi
├──tmp
└──usr
    ├──hanako
    │   └──memo.tex
    ├──tomura
    │   ├──game.doc
    │   └──junk.mail
    └──yuki
        ├──Composite.java
        └──diary.html
```

実行結果の一部

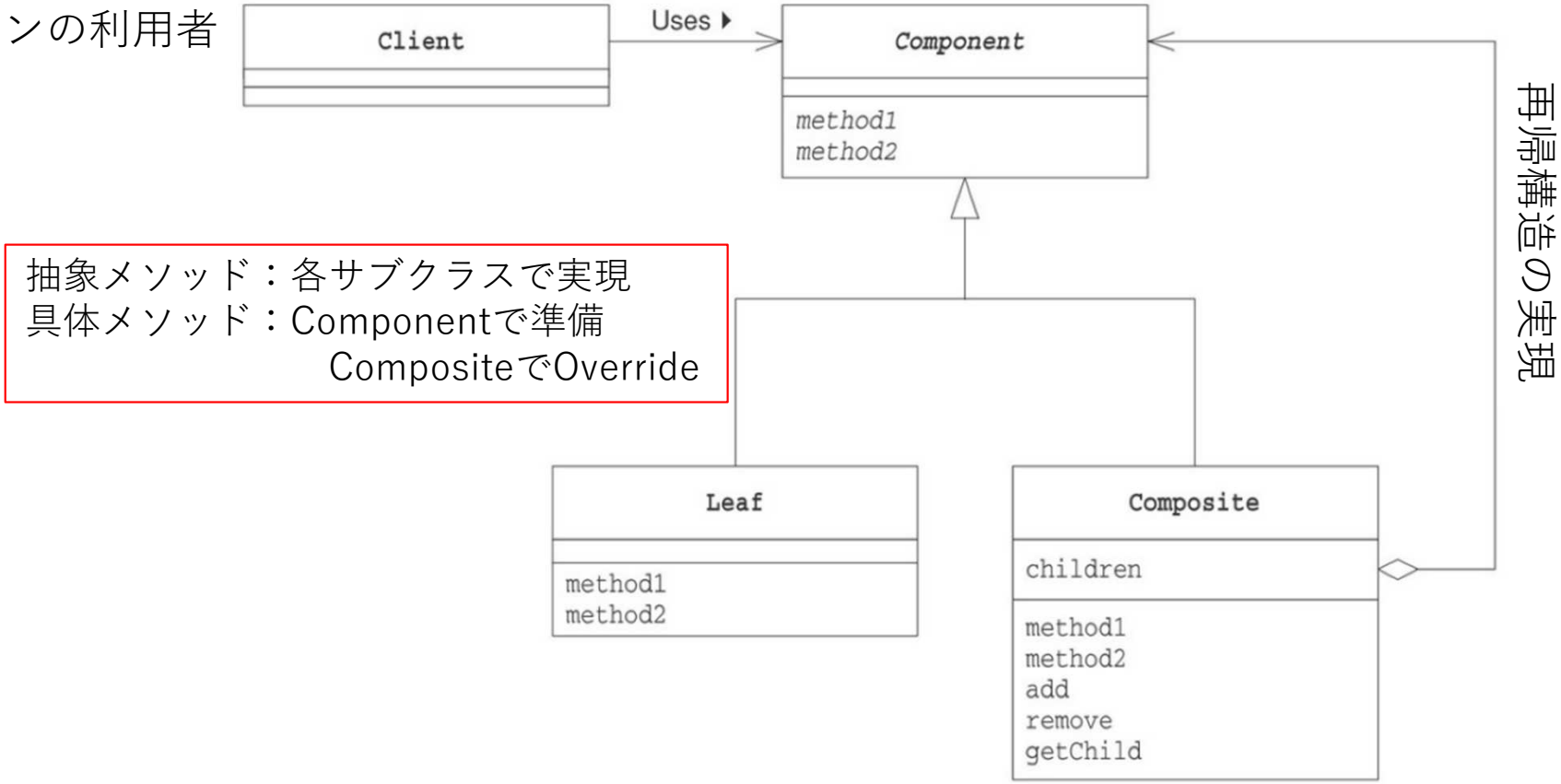
```
Making root entries...
/root (30000)
/root/bin (30000)
/root/bin/vi (10000)
/root/bin/latex (20000)
/root/tmp (0)
/root/usr (0)
...
```

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Composite パターンの登場人物

役割分担に注目しよう

- Leaf（葉）の役：Fileクラス
  - 「中身」を表す役．他のものを入れることはできない
- Composite（複合体）の役：Directoryクラス
  - 「容器」を表す役．Leaf役やComposite役を入れることができる
- Componentの役：Entryクラス
  - Leaf役とComposite役を同一視するための役．
  - Leaf役とComposite役に共通のスーパークラスとして実現する
- Client（依頼者）の役：Mainクラス
  - パターンの利用者



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考えを広げるためのヒント

- 複数と単数の同一視
  - 容器と中身を同一視する ＝『「複数」と「単数」の同一視』
    - 複数個のものを集めて，それをあたかも一つのものであるかのように扱う

- add はどこに置くべきか：サンプルプログラムではEntryに作成し，例外を投げた
  - 場合１：Entryクラスに実装し，エラーにする（サンプルプログラム）
    - 実際にaddが可能なクラスでは，オーバーライドする
  - 場合２：Entryクラスに実装し，何も行わない
  - 場合３：Entryクラスで宣言するが，実装はしない（＝抽象メソッドとして準備）
    - Fileクラスなど，本来addが不要なクラスでも，実装をしなければならない
  - 場合４：Directoryクラスだけに入れる
    - Entry型の変数をDirectory型へのキャストが必要（if文が必要となってしまう）

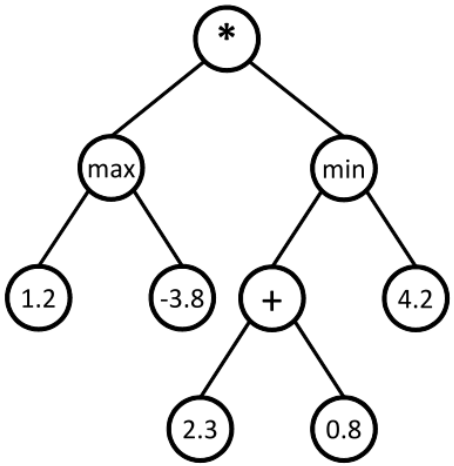
- 再帰構造が登場する場面
  - 箇条書きなど　（色々と考えてみよう）

演習2

- lec04_composite_tree/*.java
- 抽象クラスTreeElementは木の構成要素（葉，節点）を表す
  - { TreeElement，LeafElement，NodeElement }.javaの内容を確認せよ
  - { LeafElement, NodeElement }.javaに対し，3つのメソッド getHeight(), getNumberOfNodes(), contains() をそれぞれ作成せよ
  - ＜発展＞木に含まれる「葉の数」を計算するメソッドを追加せよ

  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec04_composite_tree;

- （余力のある学生さん向け）抽象クラスlec04_composite_btree/BTreeElement.javaは，2分木（の葉，節点）を表すインタフェースである．
  - BTreeLeaf は実数を一つ持つ
  - BTreePlusNode は2つの子ノードが持つ値の和を計算する
  - BTreeMultNode は2つの子ノードが持つ値の積を計算する
  - 各プログラムの内容を確認・説明せよ
  - 2つの子ノードが持つ値の{Max, Min} を計算するクラスをそれぞれ作成せよ
  - 右図の2分木の値を計算するプログラムを作成せよ
  - 指定ディレクトリ内で作成する．パッケージ宣言に注意！



- （余力のある学生さん向け）Compositeパターンを適用したプログラム例を作成せよ

Decorator パターン
～飾り枠と中身の同一視～

Decorator パターン

種々のデコレーションを統一的に扱う
デコレーションの種類を判定するためのif文は不要
　→新しいデコレーションが追加しやすい

- スポンジケーキの例
  - クリームを塗れば，何も載っていないショートケーキが出来る
  - そこにイチゴを並べれば，苺ショートケーキが出来上がる
  - さらにメッセージ付きの板チョコを載せ，蝋燭を立てると誕生日ケーキに！
- オブジェクト指向での例
  - 中心となるオブジェクトがある
  - それに「飾り付け」となる機能を一皮一皮かぶせ，より目的にあったオブジェクトに仕上げる
- Decoratorパターン
  - オブジェクトにどんどんデコレーション（飾り付け）を施すパターン
  - Compositeパターンと似ているが，
    Decoratorパターンは次々と機能追加できることに主眼を置いている
- 利用シーン
  - 透過的なインタフェースを保ったまま，次々と臨機応変に機能追加したい場合

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

サンプルプログラム

lec04_decorator/*.java

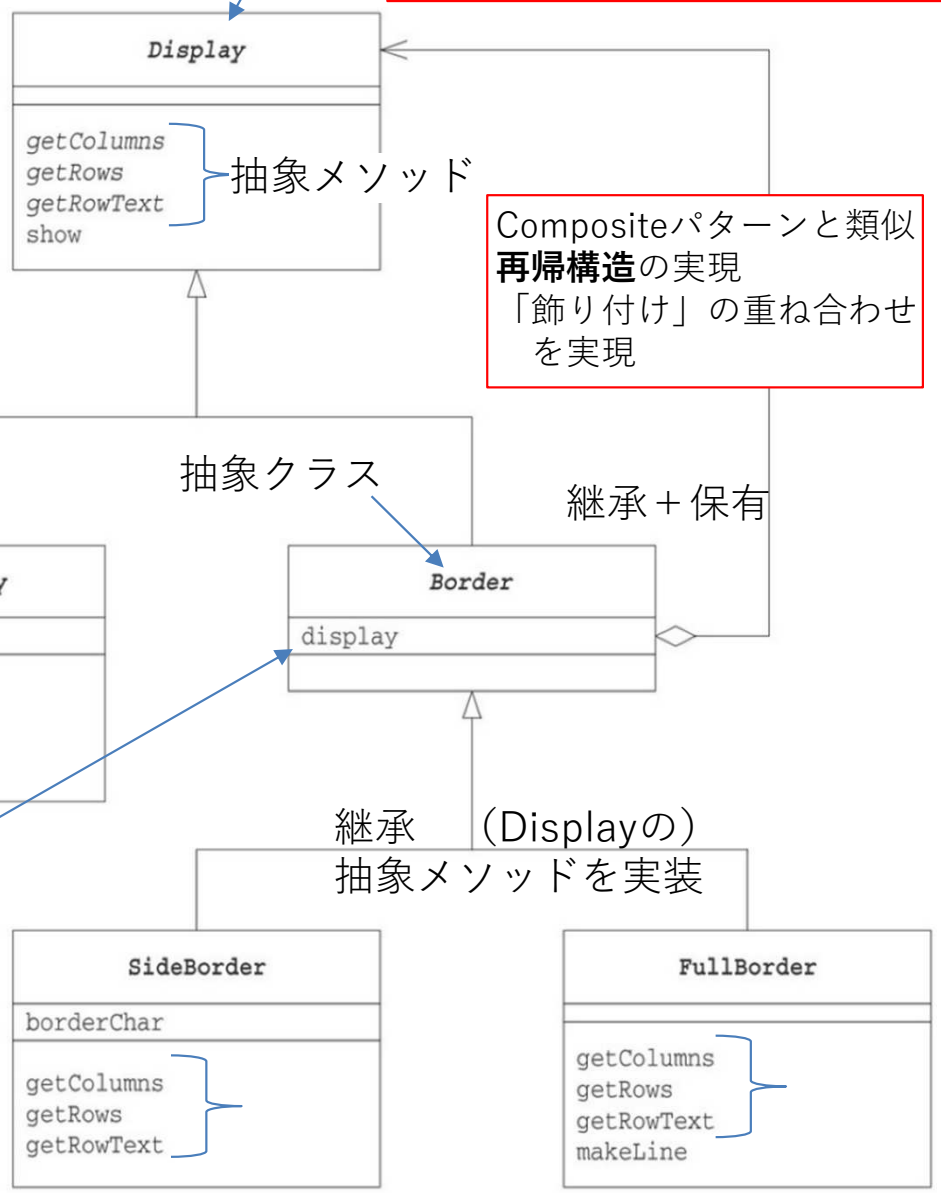
- 「文字列」の周りに飾り枠を付けて表示する
  - 飾りとは-, +, | という文字

名　前	解　説
Display	文字列表示用の抽象クラス
StringDisplay	1行だけからなる文字列表示用のクラス
Border	「飾り枠」を表す抽象クラス
SideBorder	左右にのみ飾り枠をつけるクラス
FullBorder	上下左右に飾り枠をつけるクラス
Main	動作テスト用のクラス

出力の例

```
+----------------+
|#Hello, world.#|
+----------------+
```

役割分担に注目しよう
「抽象クラス＋具体クラス」
（頻出する形です）



Borderを継承したクラス
も再帰構造を持つので，
飾り付けられたDisplayを
さらに飾ることが可能

Borderは「飾り」の基底
Borderを継承する形で
各「飾り」を準備する．
Borderは「Display」を相手にする
　＃StringDisplayではなくDisplay

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Displayクラス：（複数行からなる）文字列を表示するための抽象クラス　　lec04_decorator/*.java

抽象メソッド
（列数，行数，各行）

```
public abstract class Display {
    public abstract int getColumns();                  // 横の文字数を得る
    public abstract int getRows();                     // 縦の行数を得る
    public abstract String getRowText(int row);        // row番目の文字列を得る
    public void show() {                               // 全部表示する
        for (int i = 0; i < getRows(); i++) {
            System.out.println(getRowText(i));
        }
    }
}
```

一行ずつ表示する

すべての行を表示するメソッド（Template Methodパターンです）

```
public class StringDisplay extends Display {
    private String string;
    public StringDisplay(String string) {
        this.string = string;
    }
    public int getColumns() {
        return string.getBytes().length;
    }
    public int getRows() {
        return 1;
    }
    public String getRowText(int row) {
        if (row != 0) {
            throw new IndexOutOfBoundsException();
        }
        return string;
    }
}
```

表示対象文字列

コンストラクタ

StringDisplayクラス：
1行からなる文字列を
表示するためのクラス

抽象クラスDisplayを継承
各抽象メソッドを実装

0行数目以外ではエラーとする

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Borderクラス：飾り枠を表す抽象クラス

lec04_decorator/*.java

Displayを継承する：飾り枠は中身と同じメソッドを持つ
飾り枠（Border）と中身（Display）を同一視

```
public abstract class Border extends Display {
    protected Display display;              // この飾り枠がくるんでいる「中身」を指す
    protected Border(Display display) { // インスタンス生成時に「中身」を引数で指定
        this.display = display;
    }
}
```

コンストラクタの準備
すべての子クラスは，このコンストラクタを呼び出す必要がある

display は，飾り枠がくるむ「中身」
Displayクラスであることに注意！
Borderのインスタンスは Displayのインスタンスでもある
＝Borderのサブクラスのインスタンスも Displayのインスタンスである

Displayクラスのインスタンス display を持つことで，再帰構造が生まれる

抽象クラス＋継承＋フィールド変数 → 再帰構造
抽象クラス：サブクラスの基底となる
自分自身が具体メソッドを持つ必要はない
継承：自分自身がその型になる
フィールド変数：自分と同じ型の変数を持つ（再帰）

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

SideBorderクラス：左右に「文字」を付ける

lec04_decorator/*.java

抽象メソッドを実装する

```
public class SideBorder extends Border {
    private char borderChar;
    public SideBorder(Display display, char ch) {
        super(display);
        this.borderChar = ch;
    }
    public int getColumns() {
        return 1 + display.getColumns() + 1;
    }
    public int getRows() {
        return display.getRows();
    }
    public String getRowText(int row) {
        return borderChar + display.getRowText(row) + borderChar;
    }
}
```

左右に付ける文字

コンストラクタ
親クラスのコンストラクタを呼び出す必要あり

左右に一文字ずつ追加するので，
列数は，中身（display）の列数＋１＋１

左右に一文字ずつ追加するので，行数は変わらない

「中身の表示」は中身に任せる．

FullBorderクラス：上下左右を囲む（上下は＋--＋を，左右は｜を用いる）lec04_decorator/*.java

抽象メソッドを実装する

```
public class FullBorder extends Border {
    public FullBorder(Display display) {
        super(display);
    }
    public int getColumns() {
        return 1 + display.getColumns() + 1;
    }
    public int getRows() {
        return 1 + display.getRows() + 1;
    }
    public String getRowText(int row) {
        if (row == 0) {
            return "+" + makeLine('-', display.getColumns()) + "+";
        } else if (row == display.getRows() + 1) {
            return "+" + makeLine('-', display.getColumns()) + "+";
        } else {
            return "|" + display.getRowText(row - 1) + "|";
        }
    }
    private String makeLine(char ch, int count) {
        StringBuffer buf = new StringBuffer();
        for (int i = 0; i < count; i++) {
            buf.append(ch);
        }
        return buf.toString();
    }
}
```

コンストラクタ
親クラスのコンストラクタを呼び出す必要あり

左右に一文字ずつ追加するので，
列数は，中身（display）の列数＋１＋１

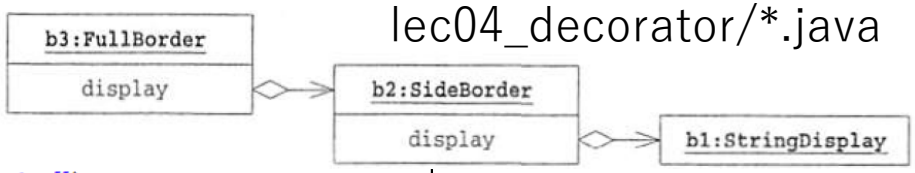
上下に一文字ずつ追加するので，
行数は，中身（display）の列数＋１＋１

最初と最後の行は　+------+
それ以外は，｜＋中身の(row-1)行目＋｜
（飾り付け後のrow行目＝中身のrow-1行目）

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Mainクラス

lec04_decorator/*.java




```
public class Main {
    public static void main(String[] args) {
        Display b1 = new StringDisplay("Hello, world.");
        Display b2 = new SideBorder(b1, '#');
        Display b3 = new FullBorder(b2);
        b1.show();
        b2.show();
        b3.show();
        Display b4 =
                new SideBorder(
                    new FullBorder(
                        new FullBorder(
                            new SideBorder(
                                new FullBorder(
                                    new StringDisplay("Decorator")
                                ),
                                '*'
                            )
                        )
                    ),
                    '/'
                );
        b4.show();
    }
}
```

b2に対する装飾（入れ子）

入れ子が深いですが，各new で一つのDisplayを作っている．
Borderとサブクラスのコンストラクタに注意し，
対応関係を確認してみよう

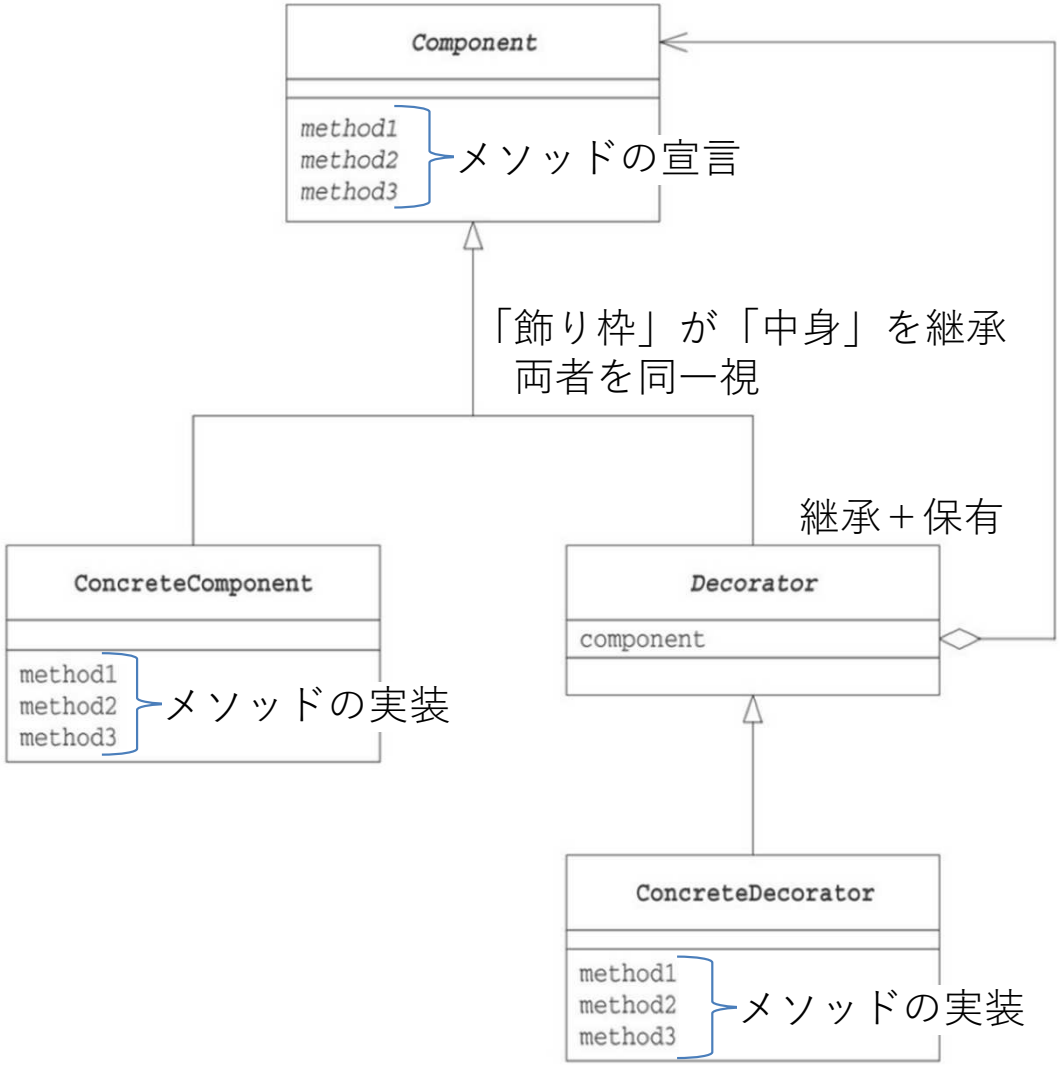
StringDisplayをFullBorderで装飾し，
それをSideBorder (*) で装飾し，
それをFullBorderで装飾し，
それをFullBorder で装飾し，
それをSideBorder (/)で装飾している
（内側から読むと分かります）

```
Hello, world.
#Hello, world.#
+---------------+
|#Hello, world.#|
+---------------+
/+---------------+/
/|+-------------+|/
/||*+---------+*||/
/||*|Decorator|*||/
/||*+---------+*||/
/|+-------------+|/
/+---------------+/
```

コンパイル方法: javac lec04_decorator/Main.java
実行方法:　　java lec04_decorator/Main

Decorator パターンの登場人物

- Componentの役：Displayクラス
  - 機能追加の核となる役．インタフェースを定める
- ConcreteComponentの役：StringDisplayクラス
  - Component役のインタフェースを実装した具体的なクラス
  - 具体的な「中身」
- Decorator（装飾者）の役：Borderクラス
  - Component役と
    同じインタフェースを持つ
  - 飾る対象となる
    Component役を持つ
- ConcreteDecorator
  （具体的な装飾者）の役：
  SideBorderクラス，
  FullBorderクラス
  - 具体的なDecoratorの役
  - 具体的な飾り枠
  - インタフェースを実装する



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考えを広げるためのヒント

- 透過的なインタフェース
  - Decoratorパターンでは，飾り枠と中身を同一視する
    - 飾り枠（Border），中身（Displayとそのサブクラス）
    - 飾り枠は，中身と同じインタフェースを持つ
  - 飾り枠をたくさん使って包んでも，インタフェースは全く変更なし
    - インタフェースが透過的
  - Compositeパターンと類似した再帰構造
    - Compositeパターン：構造を持った対象の表現が目的
    - Decoratorパターン：（再帰的な）機能の追加が目的
- 中身を変えずに機能追加可能
  - 包まれるものを変更することなく，機能を追加できる．
  - Decoratorパターンでは，委譲が利用されている
    - 「飾り枠」への要求を，中身に委譲する（中身のメソッドを利用して実現する）
- 単純な品揃えでも，多様な機能追加ができる
  - 具体的な飾り枠をたくさん用意しておけば，自由に組み合わせて新しいオブジェクトを作ることができる．
  - 個々の飾り枠は単純でも構わない
- 欠点：小さいクラスが増えてしまう
  - 良く似ている小さなクラスが多数作られてしまう場合がある

演習3

lec04_decorator/*.java

- lec04_decorator/*.java

- 以下の飾りつけを行うクラスを追加せよ.
  - 対象の上下に飾りつけを行う
  - 対象の上下左右に，一行（一文字）分の空行（空白）を空けて飾りつけを行う

- 与えられた文字に対し，種々の装飾を施して表示するプログラムを作成せよ
- 同様に，与えられた実数値に対し，種々の装飾を施して表示するプログラムを作成せよ

  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec04_decorator;

- （余力のある学生さん向け）Decoratorパターンを
  適用したプログラム例を作成せよ

実数値に対する装飾の実行例

```
#123.456 #
+----------+
|#123.456 #|
+----------+
/+--------------+/
/|+------------+|/
/||*+--------+*||/
/||*|123.456 |*||/
/||*+--------+*||/
/|+------------+|/
/+--------------+/
```